

Phase-Stratified Credit: Verifier-Free Per-Phase Advantage for Long-Context Reasoning RL

Method. Phase-Stratified GRPO (PS-GRPO)

Motivation

The user works on teaching large language models to reason over very long documents using reinforcement learning. Today’s best method, LoongRL (Long-context Reinforcement Learning), gets its results by making the training data harder – it hides the real question inside long chains of made-up key-value codes and lots of distractor text – and then trains with an off-the-shelf algorithm called GRPO (Group Relative Policy Optimization). GRPO produces one single score for the entire answer (a long chain of 16,000 to 128,000 words of plan, retrieve, reason, and double-check steps) and then applies that one score equally to every single word.

The core problem is that only a few steps in that very long chain actually decide whether the answer is right – the moments where the model traces the key chain and grounds a fact – while most of the words are just scanning through distractor text. Because the one score is spread evenly over everything, the learning signal on those few decisive steps gets diluted. The longer the document gets (toward 128,000 words), the worse this dilution becomes: the decisive steps stay a small fixed number of words while the filler grows, so the useful signal per decisive word shrinks roughly like 1 over the total length. The method collapses exactly in the long-context regime it was built for.

This is timely because LoongRL (arXiv 2025-10) is the first method to produce reasoning chains long and structured enough for this problem to even appear, and because a wave of recent work (such as CoVRL, Coupled Variational Reinforcement Learning, arXiv 2025-12) has shown you can build useful reward signals without any external grader. The key new insight is that GRPO already samples a whole batch of attempts – some right, some wrong – and the comparison between them already tells you which step made the difference, for free.

Prior work stopped short for specific reasons. LoongRL never looks back at the phase structure it creates and uses it for credit – its whole novelty is the data. KRPO (Kalman-filtered GRPO) only cleans up the single per-answer score, never breaking it into phases. GDRO (Group Distributionally Robust Optimization) only chooses which problems to train on, not where credit lands inside one answer. CoVRL changes where the reward comes from but still scores the whole chain as one unit. When this gap closes, long-context reasoning stops being limited by signal dilution, so the same data and compute can train much deeper at 128,000 words, and the phase structure LoongRL discovered becomes an active training lever instead of a passive byproduct.

Method

Background

Unchanged GRPO long-context substrate and the length-scaling evaluation that frames the contribution.

1. Build the strongest plain baseline first. Take an open large language model in the 7-8 billion-parameter range that already supports a 128,000-token context (for example Qwen2.5-7B-Instruct) and train it with the standard Group Relative Policy Optimization (GRPO) algorithm on the 16,000-token KeyChain training data. Use a simple right/wrong reward: score 1 if the correct answer text appears (after lowercasing and collapsing spaces and punctuation) inside the model’s final-answer section, otherwise 0. GRPO samples a group of 16 attempts per question, turns each attempt’s reward into one group-normalized number A_g , and spreads that single number over every token of the attempt. Save, for every attempt, its reward, its A_g , its length, and the per-token score, plus the accuracy measured all the way out to 128,000 tokens; this curve is your reference for everything that follows. *GRPO’s group-normalized scalar advantage for rollout g : the rollout reward R_g minus the group mean $\bar{R}_q$ over the std σ_q , the single scalar broadcast uniformly to all T_g tokens in the baseline.*

$$A_g = \frac{R_g - \bar{R}_q}{\sigma_q} \quad (1)$$

Why: You need the strongest plain baseline at the same compute so any gain is attributable to the redistribution, not to extra training.

2. Train the phase-stratified version on the same data for the same number of update steps as the baseline, then test both models at 16,000 / 32,000 / 64,000 / 128,000 tokens on RULER and an extended multi-hop question-answering suite (HotpotQA, MuSiQue, 2WikiMultiHopQA). Report accuracy at each length. Also measure how concentrated the credit is: for each answer call the ‘decisive’ phases the smallest group of highest-credit-density phases whose densities together first reach half of the total, then compute C, the fraction of total credit-magnitude landing on those decisive phases, and average C over all training answers at each length. This checks the central claim directly: the accuracy advantage should grow with length, and C names exactly the quantity the method is supposed to move.

Why: It checks the central claim directly: the advantage should grow with length, and C names exactly the quantity the method is supposed to move.

Verifier-Free Per-Phase Signal (Leg B)

Manufacture a trustworthy per-phase credit-density from the in-group correct/incorrect ordering and the policy’s own confidences, with no external judge or extra forward pass.

3. Add a simple, rule-based splitter that cuts each generated answer into its plan, retrieve, reason, and recheck phases. It walks through the answer once and starts a new phase whenever it hits one of the agreed boundary markers, tagging each piece with which of the four phase types it is; if an answer has no markers at all, it falls back to cutting the answer into fixed 512-token chunks and treats each chunk as one phase. You must decide what actually counts as a boundary marker. The output is, for every token, which phase number it belongs to and what type that phase is, giving S_g phases for that answer. The phases come from the model’s own observable structure, so you are not adding a new learned scorer that could be gamed.

Why: The phases come from the model’s own observed structure, so you are not adding a new learned scorer that could be gamed or that you would have to train.

- Inside the group of 16 attempts for one question, compare a correct attempt with a wrong one. Two attempts share a phase-prefix when their ordered list of phase types (plan, retrieve, reason, recheck, ...) matches from the start; you only compare the phase-type labels, not the exact words. For each correct attempt, pick the wrong attempt whose matching phase-prefix is longest (break ties by the lowest attempt id so it is reproducible), and do the same from the wrong side. The first phase where the two diverge is the decisive split: mark it +1 on the correct attempt and -1 on the wrong one, leaving every other phase at 0. If an attempt gets paired more than once, add up its +1/-1 marks across all its pairs. If the group happens to have no correct attempts or no wrong attempts, no contrast can be formed, so leave every phase at 0 for that group. The point where a right and a wrong attempt split is exactly where the decisive step lives, and the group already tells you this for free. *In-group minimal-pair contrast: a correct rollout g^+ and an incorrect g^- sharing the longest common phase-prefix split at phase s , which is credited +1 on g^+ and -1 on g^- , harvesting the ordering the binary end reward already certifies.*

$$\text{contrast}_{g^+,s} = +1, \quad \text{contrast}_{g^-,s} = -1 \quad \text{at the first phase } s \text{ where } g^+ \text{ and } g^- \text{ diverge} \quad (2)$$

Why: The point where a right and a wrong attempt split is exactly where the decisive step lives, and the group already tells you this for free.

- For each phase, take the model’s own average word-confidence — the mean log-probability the model assigned to the tokens in that phase — and reuse the confidences that were already computed when the answer was generated, so there is no extra forward pass. Then standardize these phase averages across the phases of that one answer: subtract the answer’s average-of-phases and divide by the spread across phases (add a tiny $1e-6$ to the spread so you never divide by zero). If an answer has only one phase there is no spread to compare against, so set its self-score to 0. The result is one standardized confidence number per phase. This adds a second free signal — the model’s internal confidence ordering over its own spans. *Self-score: the current policy’s mean token log-prob over phase s , z-normalized across the S_g phases of g , capturing the policy’s internal confidence ordering over its own spans with no extra forward pass.*

$$r_{g,s}^{\text{self}} = \text{z-norm}_s \left(\frac{1}{|s|} \sum_{t \in s} \log \pi_\theta(x_t \mid x_{<t}) \right) \quad (3)$$

Why: It adds a second free signal – the model’s internal confidence ordering over its own spans – without any extra forward pass.

- Blend the +1/-1 contrast (here, the +1/-1 marks meaning a phase helped or hurt the answer) and the standardized self-confidence with weights $\beta_1 = 1.0$ and $\beta_2 = 0.3$, then pass the blended per-phase values through a softmax over the phases (a softmax turns a list of numbers into positive fractions that add up to 1). This gives a credit-density d that sums to 1 across the phases. The bigger weight on the contrast means the right-versus-wrong evidence counts for more than the model’s own confidence. If a group had no contrast marks, the density just comes from the self-confidence; if there is no signal at all (a one-phase answer), the density spreads evenly across phases, which is the same as the original uniform spreading. This single number per phase is the one shared quantity both halves of the method depend on — how much of the answer’s credit each phase deserves. *Per-phase credit-density: a softmax over phases of the contrast plus the self-score ($\beta_1 = 1.0, \beta_2 = 0.3$ bridge the unit mismatch), giving the fraction of the rollout’s credit each phase should bear.*

$$d_{g,s} = \text{softmax}_s (\beta_1 \text{contrast}_{g,s} + \beta_2 r_{g,s}^{\text{self}}), \quad \sum_s d_{g,s} = 1 \quad (4)$$

Why: This single number per phase is the one shared quantity both halves of the method depend on – how much of the answer’s credit each phase deserves.

Credit-Conserving Stratified Advantage (Leg A)

Replace the uniform advantage broadcast with a phase-stratified, unbiased-in-expectation advantage that concentrates credit on the pivotal minority.

- In the training loss, replace the single uniform score with a per-phase score: every token in phase s now gets $A_{g,s} = A_g * (S_g * d_{g,s})$, where S_g is the number of phases. The S_g factor guarantees the per-phase scores still average back to the original A_g , so you neither invent nor destroy total credit — you only move it onto the phases that earned more credit-density. Leave the rest of the GRPO update (the probability-ratio term and its clipping) exactly as it was, and average the per-token losses the same way the baseline does. The effect is that the update pushes harder on the decisive phases and barely on the distractor-scanning bulk, while staying faithful to the original objective. The output is a per-token score list the same length as the answer, fed straight into the gradient step. *Phase-stratified advantage assigned to every token in phase s : the scalar A_g rescaled by $S_g \cdot d$, with the S_g credit-conservation normalizer making the redistribution unbiased in expectation over phases.*

$$A_{g,s} = A_g \cdot (S_g d_{g,s}), \quad \mathbb{E}_s[A_{g,s}] = A_g \quad (5)$$

Why: Credit is moved onto the decisive phases without inventing or destroying any total credit, so the update stays faithful to the original objective while focusing the signal.